Marten Deep Dive

Architektura zdarzeń i projekcji w .NET 10

Event Store
PostgreSQL

Runtime
.NET 10

Kompletny przewodnik po implementacji wzorca Event Sourcing

1. Zdarzenia Domenowe (Domain Events)

Zdarzenie to zapisany w czasie **fakt biznesowy**. W Event Sourcingu zdarzenia nie są skutkiem ubocznym zmiany stanu — one **SĄ** stanem.

Charakterystyka:

- **Czas przeszły:** Nazwy takie jak `PostCreated`, nie `CreatePost`.
- **Niezmienność (Immutability):** Raz zapisane zdarzenie nigdy nie ulega zmianie.
- **Brak intencji:** Zdarzenie mówi co się stało, a nie co ma się stać.
- **Minimalizm:** Przechowuje tylko "deltę" zmiany (np. tylko nowe ID i autora).

> **Wskazówka:** Używaj recordów w C#. Zapewniają one semantykę wartości i niezmienność "z pudełka".

Implementacja Zdarzeń

Spójrzmy na kompletny zestaw zdarzeń dla naszego systemu blogowego:

C# CODE

```
namespace Blog.Api.Domain.Events;

// Inicjalizacja strumienia - zawiera komplet danych startowych
public record PostCreated(
    Guid Id,
    string Title,
    string Content,
    string Author
);

// Zmiana stanu cząstkowego - tylko to, co faktycznie uległo edycji
public record PostUpdated(
    Guid Id,
    string Title,
    string Content
);

// Zdarzenie statusowe - niesie informację o czasie publikacji
public record PostPublished(
    Guid Id,
    DateTime PublishedAt
);

// Zdarzenie "negatywne" - cofnięcie stanu
public record PostUnpublished(
    Guid Id
);
```

Gdzie trafiają dane? (mt_events)

Marten nie tworzy oddzielnych tabel dla każdego typu zdarzenia. Wszystko trafia do zoptymalizowanej tabeli **mt_events** .

Struktura tabeli mt_events:

- `seq_id`: Globalny, rosnący numer sekwencyjny (BigInt).
- `stream_id`: ID naszego agregatu (np. ID Posta).
- `version`: Numer wersji wewnątrz strumienia (1, 2, 3...).
- `data`: Pełny JSON zdarzenia.
- `type`: Nazwa typu (np. "post_created").

Dzięki temu Marten może odczytać całą historię obiektu jednym zapytaniem:

```
SELECT data FROM mt_events WHERE stream_id = ? ORDER BY version
```

2. Projekcje (Read Models)

Skoro dane to historia zdarzeń, jak wyświetlić "aktualny tytuł"? Do tego służą **Projekcje** .

Typy Projekcji w Marten:

- **Inline:** Aktualizowana w tej samej transakcji co zdarzenie. Gwarantuje spójność (Strong Consistency).
- **Async:** Przetwarzana w tle przez proces Daemon. Skalowalna, ale z opóźnieniem (Eventual Consistency).
- **Live:** Budowana w pamięci tylko na czas żądania (nie zapisywana w bazie).

> **Zasada:** Zdarzenia są dla zapisu, projekcje są dla odczytu (CQRS).

Anatomia Projekcji

Poniżej znajduje się pełna implementacja `PostDetailsProjection`. Zwróć uwagę na metody `Create` i `Apply`.

C# CODE

```
public class PostDetailsProjection : SingleStreamProjection<PostDetails>
{
    // Create: Wywoływane TYLKO dla pierwszego zdarzenia w strumieniu
    public PostDetails Create(PostCreated created)
    {
        return new PostDetails(
            created.Id,
            created.Title,
            created.Content,
            created.Author,
            IsPublished: false,
            PublishedAt: null
        );
    }

    // Apply: Wywoływane dla każdego kolejnego zdarzenia, modyfikuje istniejący stan
    public PostDetails Apply(PostUpdated updated, PostDetails current)
        => current with {
            Title = updated.Title,
            Content = updated.Content
        };

    public PostDetails Apply(PostPublished published, PostDetails current)
        => current with {
            IsPublished = true,
            PublishedAt = published.PublishedAt
        };
}
```

3. Rebuilding: Elastyczność bez granic

Największa zaleta Event Sourcingu. Co jeśli po roku klient prosi o statystykę "Liczba edycji posta"?

W CRUD: Musisz dodać kolumnę `EditCount` i zgadywać dane historyczne lub zostawić puste pola.
W Marten:

1. Dodajesz pole `EditCount` do projekcji.
2. W metodzie `Apply(PostUpdated...)` dodajesz: `EditCount + 1`.
3. Uruchamiasz **Rebuild**.
4. Marten kasuje tabelę projekcji i buduje ją na nowo, czytając historię od pierwszego dnia.

C# CODE

```
// Komenda CLI / API wyzwalająca przebudowanie stanu:
await daemon.RebuildProjectionAsync<PostDetails>(cancellationToken);
```

Więcej niż SingleStream

Marten to nie tylko rzutowanie 1:1. Możemy budować znacznie bardziej zaawansowane widoki:

Multi-Stream Projections

Agregują dane z różnych strumieni w jeden dokument (np. statystyki wszystkich autorów). Używa się do tego klasy `ViewProjection`.

Flat Table (Entity)

Rzutowanie zdarzeń na płaskie tabele SQL zamiast JSON. Idealne pod raportowanie i systemy BI.

Inne zaawansowane możliwości:

- **Custom IProjection:** Pełna kontrola nad logiką projekcji i możliwość integracji z zewnętrznymi systemami (np. Redis, ElasticSearch).
- **Async Daemon:** Przetwarzanie zdarzeń asynchronicznie w tle dla maksymalnej wydajności zapisu (High Scalability).
- **Live Aggregation:** Budowanie stanu "w locie" w pamięci (z historii zdarzeń) bez zapisywania wyniku w bazie danych.

Podsumowanie

Write Side (Zapis)

- Walidacja komendy.
- Ładowanie agregatu (opcjonalnie).
- Append zdarzenia do strumienia.
- Transakcyjny zapis do mt_events.

Read Side (Odczyt)

- Automatyczna projekcja Inline.
- Szybkie zapytania po ID lub polach JSON.
- Możliwość budowania wielu widoków z tych samych danych.

Event Sourcing to nie tylko technologia, to sposób myślenia o danych jako o historii.